Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S005**
Sap ID: 60018250070	**Topic: Data Structure (Stack and Queue using Linked List)**
Branch: Artificial Intelligence & Data Science	

Experiment No. 5

Aim: To implement Stack and Queue data structures using Linked Lists to efficiently manage dynamic memory allocation for a cloud-based application that receives an unpredictable number of user requests.

Theory:

Part A: Stack Implementation using Linked List

Definition:

A stack is a linear data structure that follows the Last In First Out (LIFO) principle, where the last element inserted is the first one to be removed. All insertions (push) and deletions (pop) occur at one end called the top. Using a linked list for stack implementation allows dynamic memory allocation, meaning the stack can grow or shrink at runtime without a fixed size constraint, making it ideal for applications with unpredictable memory requirements.

Node Structure:

C Struct:

```
struct StackNode {
    int data;
    struct StackNode *next;
};
```

Stack Operations:

1. Push (Insert at Top)

Pseudocode:

```
BEGIN Push(TOP, VALUE)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.data = VALUE
    NEW_NODE.next = TOP
    TOP = NEW_NODE
    PRINT "Pushed: " + VALUE
END Push
```

2. Pop (Delete from Top)

Pseudocode:

```
BEGIN Pop(TOP)
    IF TOP == NULL THEN
        PRINT "Stack Underflow"
        RETURN NULL
    ELSE
        TEMP = TOP
        TOP = TOP.next
        VALUE = TEMP.data
        FREE TEMP
        PRINT "Popped: " + VALUE
        RETURN VALUE
    END IF
END Pop
```

3. Peek (View Top Element)

Pseudocode:

```
BEGIN Peek(TOP)
    IF TOP == NULL THEN
        PRINT "Stack is empty"
```

```
        RETURN NULL
    ELSE
        PRINT "Top element: " + TOP.data
        RETURN TOP.data
    END IF
END Peek
```

4. IsEmpty

Pseudocode:

```
BEGIN IsEmpty(TOP)
    IF TOP == NULL THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END IsEmpty
```

5. Display

Pseudocode:

```
BEGIN Display(TOP)
    IF TOP == NULL THEN
        PRINT "Stack is empty"
    ELSE
        SET TEMP = TOP
        WHILE TEMP != NULL DO
            PRINT TEMP.data + " <- "
            TEMP = TEMP.next
        END WHILE
        PRINT "NULL"
    END IF
END Display
```

Part B: Queue Implementation using Linked List

Definition:

A queue is a linear data structure that follows the First In First Out (FIFO) principle, where the first element inserted is the first one to be removed. Insertions (enqueue) occur at the rear end, and deletions (dequeue) occur at the front end. Using a linked list for queue implementation enables dynamic memory allocation, allowing the queue to handle varying numbers of requests efficiently without wasting memory or encountering overflow issues due to fixed capacity.

Node Structure:

C Struct:

```
struct QueueNode {
    int data;
    struct QueueNode *next;
};
```

Queue Operations:

1. Enqueue (Insert at Rear)

Pseudocode:

```
BEGIN Enqueue(FRONT, REAR, VALUE)
    NEW_NODE = ALLOCATE NODE()
    NEW_NODE.data = VALUE
    NEW_NODE.next = NULL
    IF FRONT == NULL THEN
        FRONT = NEW_NODE
        REAR = NEW_NODE
    ELSE
        REAR.next = NEW_NODE
        REAR = NEW_NODE
    END IF
```

```
    PRINT "Enqueued: " + VALUE
  END Enqueue
```

2. Dequeue (Delete from Front)

Pseudocode:

```
  BEGIN Dequeue(FRONT, REAR)
    IF FRONT == NULL THEN
      PRINT "Queue Underflow"
      RETURN NULL
    ELSE
      TEMP = FRONT
      VALUE = TEMP.data
      FRONT = FRONT.next
      IF FRONT == NULL THEN
        REAR = NULL
      END IF
      FREE TEMP
      PRINT "Dequeued: " + VALUE
      RETURN VALUE
    END IF
  END Dequeue
```

3. Peek (View Front Element)

Pseudocode:

```
  BEGIN Peek(FRONT)
    IF FRONT == NULL THEN
      PRINT "Queue is empty"
      RETURN NULL
    ELSE
      PRINT "Front element: " + FRONT.data
      RETURN FRONT.data
    END IF
```

```
END Peek
```

4. IsEmpty

Pseudocode:

```
BEGIN IsEmpty(FRONT)
   IF FRONT == NULL THEN
      RETURN TRUE
   ELSE
      RETURN FALSE
   END IF
END IsEmpty
```

5. Display

Pseudocode:

```
BEGIN Display(FRONT)
   IF FRONT == NULL THEN
      PRINT "Queue is empty"
   ELSE
      SET TEMP = FRONT
      WHILE TEMP != NULL DO
         PRINT TEMP.data + " <- "
         TEMP = TEMP.next
      END WHILE
      PRINT "NULL"
   END IF
END Display
```

Code - Stack Implementation:

```
#include<stdio.h>
#include<stdlib.h>
```

```
struct StackNode {
    int data;
    struct StackNode *next;
};

struct StackNode *top = NULL;

void push(int value) {
    struct StackNode *newNode = (struct StackNode*)malloc(sizeof(struct
StackNode));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("Pushed: %d\n", value);
}

int pop() {
    if(top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct StackNode *temp = top;
    int value = temp->data;
    top = top->next;
    printf("Popped: %d\n", value);
    free(temp);
    return value;
}

int peek() {
    if(top == NULL) {
        printf("Stack is empty\n");
```

```
        return -1;
    }
    printf("Top element: %d\n", top->data);
    return top->data;
}

int isEmpty() {
    return top == NULL;
}

void display() {
    if(top == NULL) {
        printf("Stack is empty\n");
        return;
    }
    struct StackNode *temp = top;
    printf("Stack: ");
    while(temp != NULL) {
        printf("%d <- ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, value;
    do {
        printf("\n--- Stack Menu ---\n");
        printf("1.Push  2.Pop  3.Peek  4.Display  0.Exit\n");
        printf("Choice: "); scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Enter value: "); scanf("%d", &value); push(value);
```

```
break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 0: break;
            default: printf("Invalid choice\n");
        }
    } while(choice != 0);
    return 0;
}
```

Output - Stack:

```
--- Stack Menu ---
1.Push  2.Pop  3.Peek  4.Display  0.Exit
Choice: 1
Enter value: 10
Pushed: 10

Choice: 1
Enter value: 20
Pushed: 20

Choice: 1
Enter value: 30
Pushed: 30

Choice: 4
Stack: 30 <- 20 <- 10 <- NULL

Choice: 3
Top element: 30
```

```
Choice: 2
Popped: 30

Choice: 4
Stack: 20 <- 10 <- NULL

Choice: 2
Popped: 20

Choice: 2
Popped: 20

Choice: 2
Stack Underflow

Choice: 0
```

Code - Queue Implementation:

```
#include<stdio.h>
#include<stdlib.h>

struct QueueNode {
   int data;
   struct QueueNode *next;
};

struct QueueNode *front = NULL;
struct QueueNode *rear = NULL;

void enqueue(int value) {
   struct QueueNode *newNode = (struct
QueueNode*)malloc(sizeof(struct QueueNode));
```

```
    newNode->data = value;
    newNode->next = NULL;
    if(front == NULL) {
        front = newNode;
        rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("Enqueued: %d\n", value);
}

int dequeue() {
    if(front == NULL) {
        printf("Queue Underflow\n");
        return -1;
    }
    struct QueueNode *temp = front;
    int value = temp->data;
    front = front->next;
    if(front == NULL) {
        rear = NULL;
    }
    printf("Dequeued: %d\n", value);
    free(temp);
    return value;
}

int peek() {
    if(front == NULL) {
        printf("Queue is empty\n");
        return -1;
```

```
    }
    printf("Front element: %d\n", front->data);
    return front->data;
}

int isEmpty() {
    return front == NULL;
}

void display() {
    if(front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    struct QueueNode *temp = front;
    printf("Queue: ");
    while(temp != NULL) {
        printf("%d <- ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, value;
    do {
        printf("\n--- Queue Menu ---\n");
        printf("1.Enqueue 2.Dequeue 3.Peek 4.Display 0.Exit\n");
        printf("Choice: "); scanf("%d", &choice);
        switch(choice) {
            case 1: printf("Enter value: "); scanf("%d", &value);
enqueue(value); break;
```

```
            case 2: dequeue(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 0: break;
            default: printf("Invalid choice\n");
        }
    } while(choice != 0);
    return 0;
}
```

Output - Queue:

```
--- Queue Menu ---
1.Enqueue  2.Dequeue  3.Peek  4.Display  0.Exit
Choice: 1
Enter value: 100
Enqueued: 100

Choice: 1
Enter value: 200
Enqueued: 200

Choice: 1
Enter value: 300
Enqueued: 300

Choice: 4
Queue: 100 <- 200 <- 300 <- NULL

Choice: 3
Front element: 100

Choice: 2
```

```
Dequeued: 100

Choice: 4
Queue: 200 <- 300 <- NULL

Choice: 2
Dequeued: 200

Choice: 2
Dequeued: 300

Choice: 2
Queue Underflow

Choice: 4
Queue is empty

Choice: 0
```

Applications:

1. Stack Applications (with dynamic memory requirements):

1. Function Call Management in Cloud Servers: Each incoming request creates a new stack frame for function execution. The number of concurrent requests varies unpredictably, requiring dynamic stack allocation to handle peak loads without wasting memory during low traffic.

2. Undo/Redo Operations in Collaborative Editing Tools: Users perform varying numbers of edit operations. A linked list-based stack dynamically grows to store action history, allowing unlimited undo levels based on available memory rather than a fixed array size.

2. Queue Applications (with dynamic memory requirements):

1. Request Handling in Load Balancers: Cloud applications receive unpredictable numbers of user requests. A linked list-based queue dynamically accommodates varying request volumes, ensuring fair FIFO processing without overflow errors or wasted pre-allocated memory.

2. Print Job Scheduling in Network Printers: Multiple users submit print jobs at varying rates. A dynamic queue manages the job list efficiently, growing as needed during peak hours and shrinking during idle periods.

Applications:

1. Stack Applications (with dynamic memory requirements):

1. Function Call Management in Cloud Servers: Each incoming request creates a new stack frame for function execution. The number of concurrent requests varies unpredictably, requiring dynamic stack allocation to handle peak loads without wasting memory during low traffic.

2. Undo/Redo Operations in Collaborative Editing Tools: Users perform varying numbers of edit operations. A linked list-based stack dynamically grows to store action history, allowing unlimited undo levels based on available memory rather than a fixed array size.

2. Queue Applications (with dynamic memory requirements):

1. Request Handling in Load Balancers: Cloud applications receive unpredictable numbers of user requests. A linked list-based queue dynamically accommodates varying request volumes, ensuring fair FIFO processing without overflow errors or wasted pre-allocated memory.

2. Print Job Scheduling in Network Printers: Multiple users submit print jobs at varying rates. A dynamic queue manages the job list efficiently, growing as needed during peak hours and shrinking during idle periods.

Conclusion:

This experiment successfully implemented both Stack (LIFO) and Queue (FIFO) data structures using linked lists. The linked list approach enables dynamic memory allocation, allowing both structures to grow and shrink at

runtime based on the number of elements. This is particularly advantageous for cloud-based applications with unpredictable request volumes, as it eliminates fixed-size constraints, prevents overflow errors, and optimizes memory utilization. All core operations (push/pop, enqueue/dequeue, peek, isEmpty, display) were implemented and tested, demonstrating O(1) time complexity for insertion and deletion operations in both structures.